

```
In [1]: def greet():  
        print('hello')  
        print('good morning team')
```

```
In [2]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()
```

hello
good morning team

```
In [3]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        def greet():  
            print('hello')  
            print('good morning team')  
            greet()
```

hello
good morning team
hello
good morning team

```
In [4]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        print()  
  
        def greet():  
            print('hello')  
            print('good morning team')  
            greet()
```

hello
good morning team

hello
good morning team

```
In [5]: def greet():  
        print('hello')  
        print('good morning team')  
        greet()  
  
        print()  
  
        def greet():  
            print('hello')  
            print('good morning team')  
            greet()  
  
        print()
```

```
def greet():
    print('hello')
    print('good morning team')
greet()
```

hello
good morning team

hello
good morning team

hello
good morning team

```
In [6]: def greet(): # declare function without argument
        print('hello')
        print('good morning team')
        greet()
        print('*****')
        greet()
        print('*****')
        greet()
```

hello
good morning team

hello
good morning team

hello
good morning team

```
In [7]: def greet():
        print('hello')
        print('good morning team')
        greet()
```

hello
good morning team

```
In [8]: # function with argument
```

```
def add(x,y):
    c=x+y
    print(c)

add(5,6)
```

11

```
In [9]: def add(x,y):
        c=x+y
        return c

add(5,6)
```

Out[9]: 11

```
In [10]: def add(x,y):
          c=x+y
          return c
```

```
add(5)
```

```
-----
TypeError Traceback (most recent call last)
Cell In[10], line 5
      2 c=x+y
      3 return c
----> 5 add(5)

TypeError: add() missing 1 required positional argument: 'y'
```

```
In [11]: def add(x,y):
          c=x+y
          return c

          add(5,6,7)
```

```
-----
TypeError Traceback (most recent call last)
Cell In[11], line 5
      2 c=x+y
      3 return c
----> 5 add(5,6,7)

TypeError: add() takes 2 positional arguments but 3 were given
```

```
In [12]: def add(x,y,z):
          c=x+y
          return c

          add(5,6,7)
```

```
Out[12]: 11
```

```
In [13]: def add(x,y,z):
          c=x+y+z+m
          return c

          add(5,6,7)
```

```
-----
NameError Traceback (most recent call last)
Cell In[13], line 5
      2 c=x+y+z+m
      3 return c
----> 5 add(5,6,7)

Cell In[13], line 2, in add(x, y, z)
      1 def add(x,y,z):
----> 2 c=x+y+z+m
      3 return c

NameError: name 'm' is not defined
```

```
In [14]: def add(x,y,z,n):
          c = x+y+z+m
          return c

          add(5,6,7,8)
```

```

-----
NameError Traceback (most recent call last)
Cell In[14], line 5
      2 c = x+y+z+m
      3 return c
----> 5 add(5,6,7,8)

Cell In[14], line 2, in add(x, y, z, n)
      1 def add(x,y,z,n):
----> 2 c = x+y+z+m
      3 return c

NameError: name 'm' is not defined

```

```

In [15]: def add(x,y,z,n):
          c = x+y+z+n
          return c

          add(5,6,7,8)

```

Out[15]: 26

```

In [16]: def greet():
          print('hello')
          print('good morning team')
          greet()

          def add(x,y):
              c = x+y
              return c

          add(5,6)

```

```

hello
good morning team

```

Out[16]: 11

```

In [17]: def greet():
          print('hello')
          print('good morning team')

          def add(x,y):
              c = x+y
              return c

          def sub(x,y):
              d = x-y
              return d

          greet()
          print(add(5,6))
          print(sub(5,6))

```

```

hello
good morning team
11
-1

```

```
In [18]: def add_sub(x,y):  
         c= x+y  
         d= x-y  
         return c, d  
  
         result = add_sub(4,5)  
         print(result)  
         print(type(result))
```

(9, -1)
<class 'tuple'>

```
In [19]: def add_sub(x,y):  
         c= x+y  
         d= x-y  
         return c, d  
  
         result, result1 = add_sub(4,5)  
  
         print(result)  
         print(result1)  
         print(type(result))
```

9
-1
<class 'int'>

```
In [20]: def add_sub_mul(x,y):  
         c= x+y  
         d= x-y  
         e= x*y  
         return c, d, e  
  
         add, sub, mul = add_sub_mul(4,5)  
  
         add  
         sub  
         mul
```

Out[20]: 20

```
In [21]: def add_sub_mul_div(x,y):  
         c= x+y  
         d= x-y  
         e= x*y  
         f=x%y  
         return c, d, e, f  
  
         add, sub, mul, div = add_sub_mul_div(4,5)  
  
         add  
         sub  
         mul  
         div
```

Out[21]: 4

UPDATE

```
In [22]: def update():
        x = 8
        print(x)
        update()
```

8

```
In [23]: def update(): #update function take the value from the user
        x = 8
        print(x)

        update(8)
```

TypeError Traceback (most recent call last)

Cell In[23], line 5

```
2     x = 8
3     print(x)
----> 5 update(8)
```

TypeError: update() takes 0 positional arguments but 1 was given

```
In [24]: def update(x): #update function take the value from the user
        x = 8
        return x

        update(100)
```

Out[24]: 8

```
In [25]: def update(x):
        x = 8
        return x

        a = 15
        update(a)
        print(a)
```

15

Positional Arguments

```
In [26]: def person(name,age):
        print(name)
        print(age)

        person('nit', 22)
```

```
nit
22
```

```
In [27]: def person(name,age):
        print(name)
        print(age)

        person('nit')
```

```

-----
TypeError      Traceback (most recent call last)
Cell In[27], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit')

TypeError: person() missing 1 required positional argument: 'age'

```

```

In [28]: def person(name,age):
          print(name)
          print(age)

          person(22)

```

```

-----
TypeError      Traceback (most recent call last)
Cell In[28], line 5
      2     print(name)
      3     print(age)
----> 5 person(22)

TypeError: person() missing 1 required positional argument: 'age'

```

```

In [29]: def person(name,age):
          print(name)
          print(age)

          person('nit', 22, 23, 45, 56)

```

```

-----
TypeError      Traceback (most recent call last)
Cell In[29], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit', 22, 23, 45, 56)

TypeError: person() takes 2 positional arguments but 5 were given

```

```

In [30]: def person(name, age):
          print(name)
          print(age)

          person(22, 'nit')

```

```

22
nit

```

```

In [31]: def person(name, age):
          print(name)
          print(age-1)

          person(22, 'nit')

```

```

22

```

```

-----
TypeError Traceback (most recent call last)
Cell In[31], line 5
      2 print(name)
      3 print(age-1)
----> 5 person(22, 'nit')

Cell In[31], line 3, in person(name, age)
      1 def person(name, age):
      2     print(name)
----> 3     print(age-1)

TypeError: unsupported operand type(s) for -: 'str' and 'int'

```

```

In [32]: def person(name, age):
          print(name)
          print(age-1)

          person(age = 22, name = 'nit')

```

```

nit
21

```

```

In [33]: def person(name, age, new_age ):
          print(name)
          print(age-1)

          person(age = 22, name = 'nit')

```

```

-----
TypeError Traceback (most recent call last)
Cell In[33], line 5
      2 print(name)
      3 print(age-1)
----> 5 person(age = 22, name = 'nit')

TypeError: person() missing 1 required positional argument: 'new_age'

```

```

In [34]: def person(name, age, new_age ):
          print(name)
          print(age-1)
          print(new_age)

          person(age = 22, name = 'nit', new_age=23)

```

```

nit
21
23

```

Default Arguments

```

In [35]: def person(name, age=18):
          print(name)
          print(age)

          person('nit')

```

```

nit
18

```



```
In [36]: def person(name, age=18):  
         print(name)  
         print(age)  
  
         person('nit', 40)
```

nit
40

Variable length Argument

```
In [37]: def person(name, age):  
         print(name)  
         print(age)  
  
         person('nit', 40, 50, 60, 70, 80 )
```

TypeError Traceback (most recent call last)

Cell In[37], line 5

```
2     print(name)  
3     print(age)  
----> 5 person('nit', 40, 50, 60, 70, 80 )
```

TypeError: person() takes 2 positional arguments but 6 were given

```
In [38]: def sum(a, b):  
         c = a+b  
         print(c)  
  
         sum(5,6,7,8)
```

TypeError Traceback (most recent call last)

Cell In[38], line 5

```
2     c = a+b  
3     print(c)  
----> 5 sum(5,6,7,8)
```

TypeError: sum() takes 2 positional arguments but 4 were given

```
In [39]: def sum(a, *b):  
         c = a+b  
         print(c)  
  
         sum(5,6,7,8)
```

TypeError Traceback (most recent call last)

Cell In[39], line 5

```
2     c = a+b
3     print(c)
----> 5 sum(5,6,7,8)
```

Cell In[39], line 2, in sum(a, *b)

```
1 def sum(a, *b):
----> 2     c = a+b
3     print(c)
```

TypeError: unsupported operand type(s) for +: 'int' and 'tuple'

```
In [40]: def sum(a, *b):
        #c = a+b
        print(type(a))
        print(type(b))

        sum(5,6,7,8)
```

```
<class 'int'>
<class 'tuple'>
```

```
In [41]: def sum(a, *b):
        c = a
        for i in b:
            c = c + i
        print(c)

        sum(5,6,7,8)
```

26

```
In [42]: def sum(a, *b):
        c = a
        for i in b:
            c = c + i
        print(c)

        sum(5,6,7,8,9,20)
```

55

```
In [43]: def sum(a, *b):
        c = a
        for i in b:
            c = c + i
        print(c)

        sum(5,6,7,8)
```

26

kwargs

```
In [44]: def person():
        person('ALEX', 36, 'JOHN', 987767)
```

```
In [45]: def person(name, *data):
        print(name)
```

```
print(data)

person('ALEX', 36, 'JOHN', 987767)
```

ALEX
(36, 'JOHN', 987767)

```
In [46]: def person(name,*data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place ='southcity', mob =987767)
```

```
-----
TypeError Traceback (most recent call last)
Cell In[46], line 5
      2     print('name')
      3     print(data)
----> 5 person('ALEX', age = 36, home_place ='southcity', mob =987767)

TypeError: person() got an unexpected keyword argument 'age'
```

```
In [47]: def person(name, **data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place ='southcity', mob =987767)
```

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767}

```
In [48]: def person(name, **data):
          print('name')
          print(data)

          person('ALEX', age = 36, home_place ='southcity', mob =987767, slary= 40000, mar
```

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767, 'slary': 40000, 'married': 'yes'}

Global & Local Variable

```
In [49]: a = 10

          def something():
              b = 15
```

```
In [50]: a = 10

          def something():
              b = 15

              print('in function',b)
              print('out function',a)
```

```
In [51]: a = 10

          def something():
              b = 15
              print('in function',b)
```

```
print('out function',a)
```

out function 10

In [52]: a = 10

```
def something():  
    a = 15  
  
    print('in function',a)  
  
    print('out function',a)
```

in function 10

out function 10

In [53]: a = 10

```
def something():  
    b = 15  
    print('in function',b)  
    something()  
  
    print('out function',a)
```

in function 15

out function 10

In [54]: a = 10

```
def something():  
  
    print('in function',a)  
  
    something()  
    print('out function',a)
```

in function 10

out function 10

In [55]: a = 10

```
def something():  
    global a  
    b = 15  
    print('in function',b)  
    print('gloabl variable', a)  
  
    something()  
  
    print('out function',a)
```

in function 15

gloabl variable 10

out function 10

In [56]: a = 20

```
def something():  
    global a  
    a = 15  
    print('in function',a)
```

```

a = 15

something()
print('out function',a)

```

in function 15
out function 15

```

In [57]: x = 10

def update_x():
    global x
    x += 5

update_x()
print(x)

```

15

```

In [58]: x = 10

def update_x():
    globals()['x'] += 5

update_x()
print(x)

```

15

Pass list to Function

```

In [59]: def count(lst):

    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
        else:
            odd +=1
    return even,odd

lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11]
even,odd = count(lst)

print(even)
print(odd)

```

5

6

```

In [60]: def count(lst):

    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1

```

```

        else:
            odd +=1
        return even,odd

lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11]
even,odd = count(lst)

print("Even : {} and odd : {}".format(even,odd))

```

Even : 5 and odd : 6

Fibonacci sequence

```

In [61]: def fib(n):
          print(0)
          print(1)

          fib(0)

```

0
1

```

In [62]: def fib(n):
          print(0)
          print(1)
          print(1)
          print(2)
          print(3)
          print(5)

          fib(0)

```

0
1
1
2
3
5

```

In [63]: def fib(n):
          a = 0
          b = 1

          print(a)
          print(b)

          for i in range(2, n):
              c = a + b
              a = b
              b = c

              print(c)

          fib(5)

```

0
1
1
2
3

```
In [64]: def fib(n):
          a, b = 0, 1
          if n == 1:
              print(a)
          else:
              print(a)
              print(b)

              for i in range(2, n):
                  c = a + b
                  a = b
                  b = c
                  print(c)

          fib(5)
```

0
1
1
2
3

Factorial number

```
In [65]: def fact(n):
          f = 1
          for i in range(1, n+1):
              f = f*i

          return f

          x = 5
          result = fact(x)
          print(result)
```

120

Recursion

```
In [67]: def wish():
          print('hello')
          wish()
```

hello

```
In [68]: def wish():
          print('hello')
          wish()
          wish()
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

24/73

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

file:///C:/Users/DELL/Downloads/Python_functions.html

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

64/73

[illegible]

```

-----
RecursionErrorTraceback (most recent call last)
Cell In[68], line 4
      2     print('hello')
      3     wish()
----> 4     wish()

Cell In[68], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[68], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

[... skipping similar frames: wish at line 3 (2972 times)]

Cell In[68], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[68], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:649, in OutStream.write
(self, string)
    646     msg = "I/O operation on closed file"
    647     raise ValueError(msg)
--> 649 is_child = not self._is_master_process()
    650 # only touch the buffer in the IO thread to avoid races
    651 with self._buffer_lock:

RecursionError: maximum recursion depth exceeded

```

```
In [69]: import sys
print(sys.getrecursionlimit())
```

3000

```
In [70]: sys.setrecursionlimit(4000)
```

```
In [71]: print(sys.getrecursionlimit())
```

4000

```
In [72]: '''
def wish():
    print('hello')
    wish()
wish()
'''
```

```
Out[72]: "\ndef wish():\n    print('hello')\n    wish()\nwish()\n"
```

```
In [73]: import sys
sys.setrecursionlimit(100)
print(sys.getrecursionlimit())

i = 0

def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()
```

```
100
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
```

```

hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75

```

```

-----
RecursionErrorTraceback (most recent call last)
Cell In[73], line 12
      10 print('hello', i)
      11 wish()
----> 12 wish()

Cell In[73], line 11, in wish()
      9 i += 1
     10 print('hello', i)
----> 11 wish()

Cell In[73], line 11, in wish()
      9 i += 1
     10 print('hello', i)
----> 11 wish()

[... skipping similar frames: wish at line 11 (72 times)]

Cell In[73], line 11, in wish()
      9 i += 1
     10 print('hello', i)
----> 11 wish()

Cell In[73], line 10, in wish()
      8 global i
      9 i += 1
----> 10 print('hello', i)
      11 wish()

File ~\anaconda3\Lib\site-packages\ipykernel\iostream.py:649, in OutStream.write
(self, string)
    646 msg = "I/O operation on closed file"
    647 raise ValueError(msg)
--> 649 is_child = not self._is_master_process()
    650 # only touch the buffer in the IO thread to avoid races
    651 with self._buffer_lock:

RecursionError: maximum recursion depth exceeded

```

Factorial using recursion

```
In [74]: def fact(n):
         if n==0:
             return 1
         return n * fact(n-1)

         result = fact(4)

         result
```

Out[74]: 24

Lambda Functions

```
In [75]: def square(a):
         return a * a

         result = square(5)
         print(result)
```

25

```
In [76]: f = lambda a: a * a

         result = f(10)
         result
```

Out[76]: 100

```
In [77]: f = lambda a, b : a + b

         result = f(1,4)
         result
```

Out[77]: 5

Filter,Map & Reduce

```
In [78]: nums = [3,2,6,8,4,6,2,9]

         evens = list(filter(is_even, nums))
```

```
-----
NameError Traceback (most recent call last)
Cell In[78], line 3
      1 nums = [3,2,6,8,4,6,2,9]
----> 3 evens = list(filter(is_even, nums))

NameError: name 'is_even' is not defined
```

```
In [82]: def is_even(n):
         return n % 2 == 0

         nums = [3,2,6,8,4,6,2,9]
         evens = list(filter(is_even, nums))
         print(evens)
```

[2, 6, 8, 4, 6, 2]

```
In [83]: def is_odd(n):  
         return n % 2 != 0  
  
         nums = [3,2,6,8,4,6,2,9]  
         odd = list(filter(is_odd, nums))  
         print(odd)
```

[3, 9]

```
In [84]: nums = [3,2,6,8,4,6,2,9]  
         evens = list(filter(lambda n : n%2 ==0, nums))  
         print(evens)
```

[2, 6, 8, 4, 6, 2]

```
In [85]: nums = [3,2,6,8,4,6,2,9]  
         odd = list(filter(lambda n : n%2 !=0, nums))  
         print(odd)
```

[3, 9]

```
In [86]: def update(n):  
         return n*2  
  
         nums = [3,2,6,8,4,6,2,9]  
         evens = list(filter(is_even, nums))  
         double = list(map(update, evens))  
  
         print(double)
```

[4, 12, 16, 8, 12, 4]

```
In [87]: nums = [3,2,6,8,4,6,2,9]  
         evens = list(filter(is_even, nums))  
         double = list(map(lambda n : n*2, evens))  
         double_ = list(map(lambda n : n-2, evens))  
         print(double)  
         print(double_)
```

[4, 12, 16, 8, 12, 4]

[0, 4, 6, 2, 4, 0]

```
In [88]: from functools import reduce  
  
         def add_all(a,b):  
             return a+b  
  
         nums = [3,2,6,8,4,6,2,9]  
  
         evens = list(filter(is_even, nums))  
         double = list(map(lambda n : n*2, evens))  
         sums = reduce(add_all, double)  
  
         print(sums)
```

56

```
In [89]: from functools import reduce  
  
         def add_all(a,b):  
             return a+b
```

```

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
sums = (reduce(add_all, double))
print(evens)
print(double)
print(sums)

```

```

[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
56

```

In [90]: `from functools import reduce`

```

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))
sums = (reduce(lambda a,b : a + b, double))

print(evens)
print(double)
print(sums)

```

```

[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
56

```

Python Decorators

In [91]: `def div(a,b):`
 `print(a / b)`
`div(4,2)`

2.0

In [92]: `def div(a,b):`
 `print(a / b)`
`div(2,4)`

0.5

In [93]: `def div(a,b):`
 `if a<b:`
 `a,b = b,a`
 `print(a / b)`
`div(2,4)`

2.0

In [94]: `def div(a,b):`
 `print(a / b)`
`def div_decorator(func):`
 `def inner(a,b):`
 `if a<b:`
 `a,b = b,a`
 `return func(a,b)`
 `return inner`


```
div = div_decorator(div)

div(2,4)
```

2.0

```
In [95]: def my_decorator(func):
          def wrapper():
              print("Something is happening before the function is called.")
              func()
              print("Something is happening after the function is called.")
          return wrapper

          @my_decorator
          def say_hello():
              print("Hello!")

          say_hello()
```

```
Something is happening before the function is called.
Hello!
Something is happening after the function is called.
```

In []: